PyOmniGraph - Getting and hosting your own federated copies of subgraphs of Wikidata and other knowledge graphs*

Wolfgang Fahl[1,*], Tim Holzheim[1], Christoph Lange[2] and Stefan Decker[1]

[1]RWTH Aachen University, Computer Science i5, Aachen, Germany
[2]Fraunhofer FIT, Sankt Augustin, Germany

Abstract

Wikidata is an example for a very large, crowd sourced general knowledge graph backed by a world wide community since 2012. Factgrid is using the same Wikibase infrastructure as Wikidata since 2017 for historical research projects. GOV (Geschichtliches/Genealogisches Ortsverzeichnis) is an online gazetteer for searching ancient places. Wikidata grew so large that the Blazegraph endpoint powering it was pushed to the 4 Terrabyte limit and the Wikimedia Foundation therefore decided to split the knowledge graph to host two instances. The challenges of federated queries that came up in the above use cases led to the need of being able to work with subgraphs of the RDF graphs and try out different alternative Endpoints such as QLever and Virtuoso and others. While typical research projects that do such subgraphing and benchmarks roll their own environments we decided to provide a python based Open Source library pyomnigraph to provide a unified infrastructure with a standard set of REST APIs and CLI commands to improve the handling and potentially get a bigger matrix of subgraphs versus endpoints to work with. In this work we report on pyomnigraph usage on a matrix of three RDF graphs: Wikidata, Factgrid and GOV and endpoints blazegraph, jena, Qlever on a selection of subgraph combinations. We show that typical frustrations of scaling multi-hop federated queries can be avoided if the subgraphs are locally loaded and federation is applied locally.

Keywords

Wikidata, FactGrid, QLever, Blazegraph, Apache Jena, GOV, RDF triple store, federated SPARQL, benchmark, PyOmniGraph

1. Introduction

Wikidata [1] started in 2012 as an effort to back the different international Wikipedia sites with a common set of facts. Initially, about 2 million entities were harvested from existing Wikipedia content. As of 2016, over 100 million entities have been curated by a growing worldwide community of contributors. In 2022, we reported on How to get and host your own copy of Wikidata [2]. For quite a few use case the need for federated queries based on subgraphs of diverse public and internal knowledge-graphs has been our motivation to try out different combinations of endpoints and subgraphs. In the case of Wikidata we even have been forced to do so by the 2025 graphsplit [3].

Public alternatives to the official Wikidata Query Service WDQS are still rare. Our attempts to convince the Wikimedia Foundation to support a public Mirror infrastructure have so far been in vain. Table 1 lists the seven workin public Wikidata endpoints currently configured in nicescholia[1] to be watched as potential alternative scholia backends.

The Wikimedia Foundation itself now serves three: the legacy full graph[2] and, since the WDQS graph

6th Wikidata Workshop 2026, co-located with ISWC 2026, October 25–26, 2026, Bari, Italy

* Preprint. Intended for submission to the 6th Wikidata Workshop 2026 (co-located with ISWC 2026); if accepted, to appear in the CEUR-WS proceedings. This version has not been peer-reviewed.

*Corresponding author.
wf@bitplan.com (W. Fahl)
0000-0002-0821-6995 (W. Fahl); 0000-0003-2533-6363 (T. Holzheim); 0000-0001-9879-3827 (C. Lange); 0000-0001-6324-7164 (S. Decker)

[1]https://nicescholia.wikidata.dbis.rwth-aachen.de/api/endpoints
[2]https://query-legacy-full.wikidata.org/sparql

Table 1
Public Wikidata query endpoints

Provider	Triple Store	Triples (B)	Last update
Wikimedia Foundation (legacy full)	Blazegraph	22.0	continuous
Wikimedia Foundation (main)	Blazegraph	8.6	continuous
Wikimedia Foundation (scholarly)	Blazegraph	8.7	continuous
RWTH Aachen DBIS	Blazegraph	17.0	2025-06-01
RWTH Aachen DBIS	QLever	–	continuous
University of Freiburg	QLever	–	continuous
OpenLink Software (truthy)	Virtuoso	–	2025-11-01

split, a main[3] and a scholarly[4] half. RWTH Aachen DBIS hosts a full Blazegraph mirror[5] alongside a QLever endpoint[6]; the University of Freiburg QLever[7] service has moved to the `qlever.dev` host. OpenLink Software[8] provides a Virtuoso instance restricted to truthy statements. These alternatives still have their own limitations regarding SPARQL language coverage, data currency and completeness.

Using federated queries on participants of the Linked Open Data Cloud LOD Cloud[9] is in theory an excellent way to answer queries over diverse data sources. In practice there are a lot of possible frustrations involved such as outages of endpoints, incompatibilities of endpoints, awkward syntax and general query rot.

For real life use cases only that operate on small subgraphs it should be feasible to operate locally by providing the necessary data "on the fly". Unfortunately to do so a lot of technical expertise is needed to set up endpoints, collect the subgraph data and finally perform the query. To ease this pain we introduce pyomnigraph which mitigates the complexity by using docker and python to provide a unified interface to different endpoint technologies in a local environment with limited resources.

The contributions of this work are:

- **pyomnigraph**, an Apache-2.0 licensed Python library that unifies the deployment and operation of multiple RDF triple stores behind a single API, REST interface and Docker based setup (Section 3), covering the backends listed in Section 3.1 and the `omnigraph` and `rdfdump` command-line interfaces (Section 3.2).

2. Related Work

Willighagen et. al. [3] report on the three options that have been considered for surviving the 2025 Wikidata graph split: Federated queries between the main and the scholar graph, using qlever with modified queries on a full graph that does not suffer from the 4 TB limit of the blazegraph endpoint or using snapquery with named parameterized queries as a middle ware to separate the endpoints concerns from the domain needs.

Linked Data Sets

CONSTRUCT queries in SPARQL have been studied by Kostylev et al. [4].

3. The pyomnigraph Resource

pyomnigraph provides a unified python interface for multiple graph databases. The software is open source under Apache 2.0 license and hosted on github.

[3]https://query-main.wikidata.org/sparql
[4]https://query-scholarly.wikidata.org/sparql
[5]https://wdqs.wikidata.dbis.rwth-aachen.de/sparql
[6]https://qlever-api.wikidata.dbis.rwth-aachen.de/
[7]https://qlever.dev/api/wikidata
[8]https://truthy.demo.openlinksw.com/sparql
[9]http://cas.lod-cloud.net/

Table 2
Supported triple store backends per the pyomnigraph README as of 2026-07-31.

Database	Status	Notes
Apache Jena	working	Robust, standards compliant
Blazegraph	working	High performance, easy setup
Oxigraph	working	Rust-based, embedded, fast
QLever	working	Extremely fast queries
GraphDB	disabled	License required (free version available)
MillenniumDB	disabled	Import-first workflow, Property Graph + RDF
Virtuoso	disabled	Permissions issue
Stardog	planned	License required

pyomnigraph leverages the named parameterized query approach from the snapquery [5] framework, which handles such parameterized SPARQL queries. It supports semantic annotation of queries and monitoring of their execution. By abstracting away the underlying endpoint, it can call multiple endpoints at once to compare results, or swap endpoints transparently, without users noticing or having to change anything.

The same separation of concerns—hiding details of endpoints and queries—is used by pyomnigraph. For human readability you will find YAML versions of these details in the `examples/<usecase>/` folders of our paper repository.

3.1. Supported Backends

Table 2 reproduces the supported triple stores table of the pyomnigraph README[10].

3.2. Command-Line Interfaces

There are two command line interfaces available for pyomnigraph: omnigraph for server management and rdfdump for data operations.

4. Use cases and Data Sources

Three use cases are the basis for our demonstration and evaluation: papers and authors (scholarly), historical locations for genealogy research and railway journey planning. For each use case we specify a list of information needs from which we derive concrete examples queries. The abstraction is then done applying named parameterized queries.

Table 3 shows the subgraph sources for the usescases.

Table 4 tracks the named parameterized queries that we use to demo and benchmark, one row per query and example binding. Each query is defined in `examples/<usecase>/queries.yaml` with its parameter list and default values, each endpoint in `examples/<usecase>/endpoints.yaml`, so a row is reproduced by a single `sparqlquery` call; the `query.sh` script of each use case reruns all its queries and writes a `<QueryName>.md` result file per query.

The examples are reproducible from the pyomnigraphPaper github repository[11]; the result sets that do not fit the paper are available in the query issues of the repository.

As typical for real world use cases the ideal persistent identifiers for linking have limited availability. ORCID and ROR would be such identifiers for the scholarly case, according to Jesper Zedlitz wäre für GOV ein Abgleich über die aktuellen Gemeinden in Deutschland. Das sollte über den Amtlichen Gemeindeschlüssel (P439) gut funktionieren.

[10]https://github.com/WolfgangFahl/pyomnigraph
[11]https://github.com/WolfgangFahl/pyomnigraphPaper

Table 3
Public SPARQL endpoints serving the subgraph sources, probed 2026-07-31 with `SELECT * WHERE {?s ?p ?o} LIMIT 1`.

Dataset	Endpoint	Engine	Probe
dblp	qlever.dev/api/dblp	QLever	0.072 s
GND	sparql.dnb.de/api/gnd	QLever	0.153 s
Wikidata	query.wikidata.org/sparql	Blazegraph	0.355 s
FactGrid	database.factgrid.de/sparql	Blazegraph	0.100 s
OpenStreetMap	sophox.org/sparql	Blazegraph	0.129 s
OpenStreetMap	qlever.dev/api/osm-planet	QLever	0.619 s
ERA RINF	rinf.data.era.europa.eu/api/sparql	RDF4J	0.139 s
GOV	gov-sparql.genealogy.net/dataset/sparql	Jena	0.060 s

Table 4
Named parameterized queries per use case, with the example parameter binding used for the reproduction run and the resulting number of records. Run with `sparqlquery` against the endpoint declared in `examples/<usecase>/endpoints.yaml`

Use case	Query	Dataset	Example	Records
scholarly	AuthorPapers	dblp	27/6858 (Fahl)	6
scholarly	CoauthorPapers	dblp	FahlHW0D22a (limit 10 of 600)	10
scholarly	EventAuthorPapers	Wikidata	Q133307039 (limit 10 of 233)	10
scholarly	InstitutionAuthorPapers	GND	36225-6 (limit 10 of 27)	10
locations	PlaceIdentity	GOV	AACHENJO30BS (Aachen)	6
railway	RelationStops	OpenStreetMap	10492086 (MD 18061)	18

4.1. scholarly - authors and papers

Queries: the papers of an author, the papers of all coauthors of a given paper, the papers of all authors of a given scientific event, the papers of all authors of a given institution

4.2. historical locations

For the the historical location gazetteer GOV [6] a List of Queries[12] has been curated which need to be extended by references to FactGrid and Wikidata.

4.3. railway journey planning

This usecase is inspired by our velorail[13] work as presented

5. Federated Query Benchmark

6. Evaluation

7. Availability

8. Conclusion and Future Work

Declaration on Generative AI

AI assistance in this project is limited to spelling, grammar, and formatting/build tooling; AI is NOT used to generate scientific content, arguments, or prose.

[12]https://gov-wiki.genealogy.net/index.php?title=List_of_Queries
[13]https://github.com/WolfgangFahl/velorail

References

[1] D. Vrandečić, M. Krötzsch, Wikidata: A free collaborative knowledgebase, Communications of the ACM 57 (2014) 78–85. doi:10.1145/2629489.

[2] W. Fahl, T. Holzheim, C. Lange, A. Westerinen, S. Decker, Getting and hosting your own copy of Wikidata, in: Proceedings of the 3rd Wikidata Workshop 2022, volume 3262 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2022. URL: https://ceur-ws.org/Vol-3262/paper9.pdf, wikidata Q115055567.

[3] E. L. Willighagen, D. Mietschen, P. Patel-Schneider, K. Linden, J. Kalmbach, L. G. Willighagen, W. Fahl, H. Bast, Scholia 2026: Compliance with SPARQL 1.1, in: SWAT4HCLS 2026: The 17th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences, March 23–26, 2026, Amsterdam, The Netherlands, SWAT4HCLS, 2026, pp. 1–3. URL: https://hdl.handle.net/2066/331667.

[4] E. V. Kostylev, J. L. Reutter, M. Ugarte, CONSTRUCT queries in SPARQL, in: 18th International Conference on Database Theory (ICDT 2015), volume 31 of *Leibniz International Proceedings in Informatics (LIPIcs)*, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2015, pp. 212–229. doi:10.4230/LIPIcs.ICDT.2015.212.

[5] W. Fahl, T. Holzheim, C. Hoyt, D. Priskorn, et al., SnapQuery: FAIR Management of Query Sets to Mitigate Query Rot in Knowledge Graphs, 2026. doi:10.5281/zenodo.17802424.

[6] J. Zedlitz, N. Luttenberger, Modelling (historical) administrative information on the semantic web, in: WEB 2014: The Second International Conference on Building and Exploring Web Based Environments, April 20–24, 2014, Chamonix, France, IARIA, 2014, pp. 33–39. URL: https://www.thinkmind.org/index.php?view=article&articleid=web_2014_2_30_40037.